

A Snake Stack

Vadlamudi Lokesh
Bharat Chandra

April 2026

Contents

1	Project Overview	2
2	Installation and Running	2
3	Webpage Layout	2
4	External Libraries Used	4
4.1	Flask	4
4.2	csv	4
5	Features Implemented	4
5.1	Home Screen	4
5.2	Snake movement	4
5.3	Game Frame Rate	5
5.4	Score, Snake Length and Snake Health	5
5.5	Immunity	5
5.6	Game Layout	5
5.7	Food Types and Spawning	6
5.8	End Screen	6
5.9	Obstacles	7
5.10	Administration	8
6	Directory structure	9
6.1	Short description of each file	9
7	Three Layer Communication	10
8	Design and Architectural Decisions	10
9	Retrospection	11
9.1	Problems faced and how we overcame	11
9.2	What could have been done if we had 10 more hours	11
10	Bibliography and references	11

1 Project Overview

This project implements a browser-based Snake game with a full three-layer architecture: a JavaScript frontend that the player interacts with, a Python (Flask) backend that receives and stores game results, and a Bash administration script for inspecting and managing stored data from the terminal.

Each layer has a single, well-defined responsibility and does not reach into the concerns of another layer. JavaScript does not know how scores are saved. The Bash script does not know how the game works. Flask sits in between and connects them.

2 Installation and Running

Install Python 3.10 if it has not been installed and run the following commands in your terminal

```
git clone https://github.com/Secreton456/A-Snake-Stack.git A_Snake_Stack
cd A_Snake_Stack
python3.10 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python3 app.py
```

Now go to <http://127.0.0.1:5000/> on your browser to play the game.

3 Webpage Layout

- The webpage loads to a home screen where the user is prompted to enter a valid username and choose a difficulty level.
- A rules section is also available where users can view the game rules.

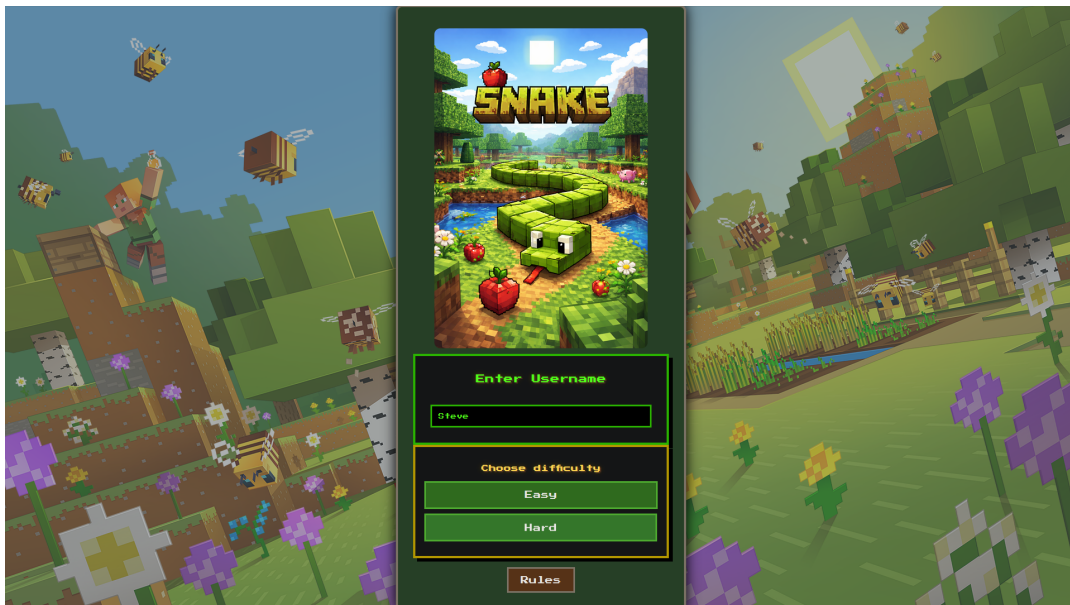


Figure 1: A Screenshot of the Home Screen

- Once a difficulty is selected, the user enters the in-game screen and can start playing.



Figure 2: A Screenshot of the in-game Screen in Hard Difficulty

- Once the player dies for whatever reason, an end screen pops up displaying a humorous death message, cause of the death, Final Score, the time survived, and a humorous status.
- The user can choose to return to the home screen or continue playing with the same username and difficulty.

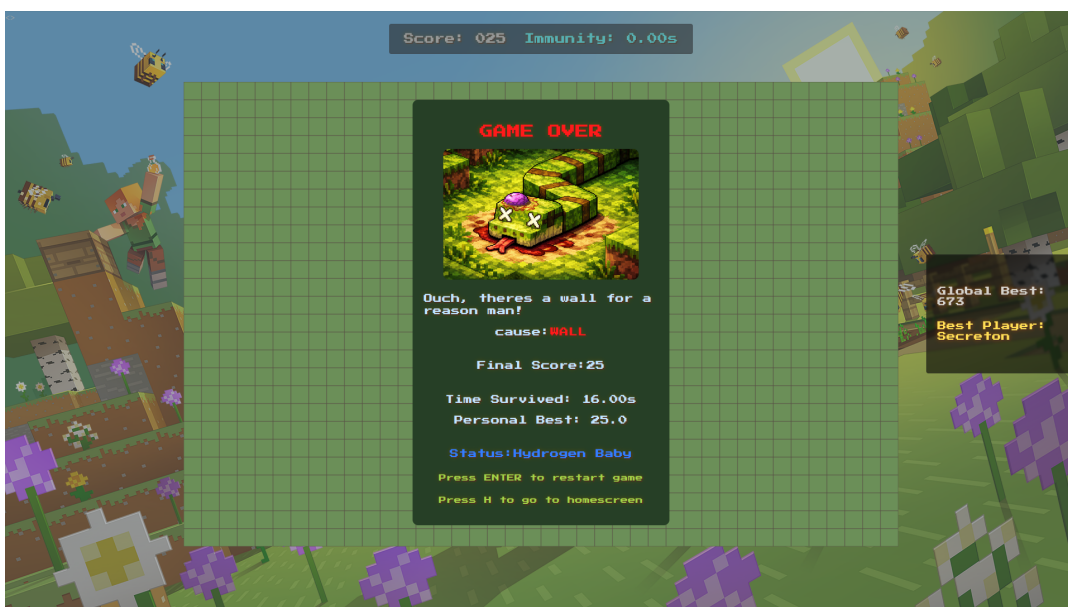


Figure 3: A Screenshot of the end screen

4 External Libraries Used

4.1 Flask

Flask [2] is a lightweight WSGI web application framework. It is designed to make getting started quick and easy, with the ability to scale up to complex applications.

- Flask Version Used: 3.1.3
- Python Version Used: 3.10

In this project Flask is majorly used for three purposes:

- The flask server serves `index.html` when the user visits the URL (/).
- Upon the player's death, the Flask server retrieves the game state from the frontend as a string, validates it, parses it into JSON and appends it to `history.txt`.
- Whenever the player dies or the webpage loads, the flask server iterates through all the logs present in `history.txt` to evaluate the highest score among all users and post the highest score and the user with that score to the frontend.

4.2 csv

Python's `csv` [1] module is a built-in library used for reading and writing tabular data in Comma Separated Values (CSV) format. It handles the complexities of CSV parsing, such as properly managing different delimiters, quoting characters, and newline formats.

This project uses the built-in `csv` module provided by Python 3.10.

In this project the `csv` Library is used for reading through all the log files when evaluating the highest score among all logs present in `history.txt`.

5 Features Implemented

5.1 Home Screen

A modal popup appears when the webpage loads, where

- The player must choose a difficulty.
- The player is prompted to enter a username, which is then validated. The username must not contain any special characters to prevent parsing errors when the Flask server processes the fetched data.
- A rules section is also available, which when clicked, pops a modal where the player can read the rules.

5.2 Snake movement

- Movement is controlled using WASD or Arrow keys, implemented through `keydown` event listeners that capture user input in real time.
- All snake cells are maintained in an array, which is rendered on the screen at every frame of the game. During each update cycle, the last cell is removed, and a new head cell is added in the direction of the current movement, effectively simulating the snake's motion.

5.3 Game Frame Rate

- A variable `timestamp` stores the elapsed time since the `gameLoop` function was last invoked. The `gameLoop` itself is repeatedly called using JavaScript's inbuilt `requestAnimationFrame`, ensuring smooth and frame-synchronised updates.
- The variable `basemovementrate` governs the overall update frequency of the game engine, effectively controlling how often the game state is refreshed. In contrast, `snakemovementrate` specifically determines how frequently the snake advances within the game grid.
- This separation allows dynamic adjustment of the snake's speed without affecting the core game loop. For instance, when the snake interacts with obstacles such as SoulSand or BlueIce, `snakemovementrate` is modified to simulate slowed or accelerated movement, while the base engine continues running at a consistent rate.

5.4 Score, Snake Length and Snake Health

In this game $\text{Score} = \text{Snake Health} \neq \text{Snake Length}$.

- Snake Length increases by one whenever a food item is eaten.
- Snake health/Score of the player changes based on the food consumed or the effects of obstacles encountered.
- Various food items increase the Score/Snakehealth by a varied amount (refer to Table 1).
- Various obstacles change the Score/SnakeHealth.

5.5 Immunity

- Certain food items provide immunity to the snake for a certain duration (refer to Table 1).
- When the snake is immune, it remains unaffected from all negative effects i.e. its score and health do not decrease, and it cannot die from colliding with walls or burning in lava.

5.6 Game Layout

5.6.1 Explanation

- A centrally placed HTML5 Canvas Element for the game window (refer to Figure 2).
- A Scoreboard displayed at the top of the screen displaying, the current score, the duration of immunity left for the player, Overall highscore among all players, The username of the player with the highest score.

5.6.2 Implementation

- A `<canvas>` element is defined in `index.html`, and its 2D rendering context is initialized in `logic.js`. The `UpdateCanvas` function is responsible for drawing the game state and is called within the `gameLoop`, which runs once per game frame.
- The Scoreboard is updated every 0.25s with the help of `updateTimers` function, which in turn is called in `gameLoop`.

5.7 Food Types and Spawning

5.7.1 Explanation

- Various foodtypes have been implemented as listed below:

FOOD TYPE	HEALTH	IMMUNITY(in s)
APPLE	1	0
CARROT	1	0
PUMPKIN PIE	4	0
GOLDEN CARROT	5	5
GOLDEN APPLE	5	7
ENCHANTED APPLE	10	10

Table 1: Various foodtypes and their functions

5.7.2 Implementation

- A Food class has been implemented and 6 objects have been created of this class with each object having different attributes, The objects are stored in a map and are accessed whenever required.

```
class Food(){
  constructor(name, health, immunity, img_src){
    //LOGIC HERE
  }
}
FOOD_ITEMS = new Map([
  ["APPLE", new Food("APPLE", 1, 0, 0.4, "/static/assets/apple.png")],
  ...
])
```

- During every frame of the game all the grid cells are iterated over and a randomly generated number is assigned to each cell of the grid using `Math.random()` in javascript, and if this number exceeds a certain value say p , then a food item is spawned in that cell.
- p is inversely proportional to the exponent of the number of food items already present on the grid i.e. $p \propto e^{-N}$ where N is the number of food items already present on the grid. Refer to the simplified code below from `logic.js`.

```
for (let i = 0; i < CANVAS_WIDTH / GRID_WIDTH; i++)
  for (let j = 0; j < CANVAS_HEIGHT / GRID_WIDTH; j++)
    if (
      Math.random() < (object.probability) / (e ** (Food_cells.length))
    )
      Food_cells[Food_cells.length] = { x: i, y: j, type: food_name };
```

5.8 End Screen

- Whenever the snake dies due to a collision with a wall, its own body, lava, or when its health reaches zero, an endscreen modal pops up displaying a humorous death message, score of

the player, cause of the death, Time survived, and an option to either continue playing or go back to home screen.

- A POST[3] request is sent to the flask server at `/save_score` with the current game data and the flask server parses the incoming JSON data into a dictionary and appends it to `history.txt`, Refer to the code from `logic.js` and `app.py`

```
fetch("http://127.0.0.1:5000/save_score", {
  method: "POST",
  body: JSON.stringify({
    // data
  })
})

@app.route("/save_score", methods=["POST", "GET"])
def get_score():
    data = request.get_json()
    with open(history, "a") as file:
        // write
    return message
```

- Another POST request is sent to the flask server with game data at `/highscore.json` to fetch the Global highscore and best player, the flask server parses the incoming JSON and iterates over all the log files in `history.txt` to get the player with the highest score using a csv reader.

5.9 Obstacles

5.9.1 Explanation

- Various obstacles with different effects have been implemented as listed below:

OBSTACLE	EFFECT
ROTTEN FLESH	The Score goes down by 1 for 5 seconds after consumption.
LAVA	Immediate death of the player.
MAGMA	Reduction of score by 0.5 every frame the snake is in contact with the block.
SOUL SAND	Slows down the snake by half of its default speed.
BLUE ICE	Speeds the block by twice its default speed.

Table 2: Caption

5.9.2 Implementation

- A super class `Obstacle` is made with sub classes inheriting this class.
- Subclasses `RottenFlesh`, `Lava`, `Magma`, `SoulSand`, and `BlueIce` inherit from the superclass constructor along with the methods `isActive` and `isPassiveActive`. Each subclass implements its own `effect` function, which defines the specific behavior or impact associated with that entity.

```
class Obstacle(){
  constructor(name, row, column, img_src){
    // CODE
```

```
    }  
}  
class RottenFlesh extends Obstacle(){  
    constructor(){  
        super();  
    }  
    // CODE  
}
```

- The spawning of obstacles follows a similar approach to food items, with one key difference: a new instance of the corresponding subclass is created each time an obstacle spawns. In contrast, food items are retrieved from a predefined lookup map of existing objects.

5.10 Administration

5.10.1 Explanation

`admin.sh` is used to manage and administer the logs stored in `history.txt`. The administrator can perform the following operations:

- View user-specific statistics, including recent games or overall metrics such as mean score, average survival time, and the distribution of different death types.
- Sort log entries based on timestamp, username, or score.
- Delete entries based on username, timestamps, or invalid record formats in `history.txt`.
- Perform log rotation by compressing `history.txt` and storing archived logs in the `archive/` directory.
- View all the unique usernames present in the log file.

5.10.2 Implementation

- User-specific statistics are computed using `awk` by parsing each log entry and filtering based on the username.
- Sorting of log entries is performed using the built-in `sort` utility in Bash.
- Deletion of log entries is handled using a combination of `sed` and `awk`.
- Log rotation is performed using `logrotate`. The file `administration/logrotate.conf` is dynamically generated by `admin.sh`, which first checks whether `history.txt` exceeds 10KB. Upon rotation, older logs are compressed and stored in the `archive/` directory, retaining up to four archived files while preserving the last 10 lines in `history.txt`. The file `administration/logrotate.state` maintains state information to prevent redundant or incorrect rotations.
- Extraction of all unique usernames is performed using `awk`.
- Viewing all unique usernames is done with the help of `awk`.

6 Directory structure

```
.
|-- administration
|   |-- deletebytimestamp.awk
|   |-- logrotate.conf
|   |-- logrotate.state
|   '-- userstats.awk
|-- admin.sh
|-- app.py
|-- archive
|   |-- history.txt.1.gz
|   |-- history.txt.2.gz
|   |-- history.txt.3.gz
|   '-- history.txt.4.gz
|-- history.txt
|-- README.md
|-- requirements.txt
'-- static
    |-- assets
    |   '-- fonts
    |-- index.html
    |-- logic.js
    '-- styles.css
```

6.1 Short description of each file

- **administration/deletebytimestamp.awk**: Implements filtering logic for deleting entries in **history.txt** based on the specified timestamps.
- **administration/logrotate.conf**: Configuration file for rotating **history.txt**; initially empty and populated dynamically by **admin.sh**.
- **administration/logrotate.state**: Maintains logrotate's state information (e.g., last rotation times) to prevent redundant or incorrect rotations of **history.txt**.
- **administration/userstats.awk**: Calculates various user-specific stats like Mean Score, Mean Time Survived, fraction of specific death types.
- **admin.sh**: Handles the entire administration of **history.txt** by using the files in **administration** directory.
- **app.py**: Handles entirety of backend logic.
- **archive/**: Stores upto 4 compressed log files after performing logrotation.
- **history.txt**: Stores log files of the game data in specific format.
- **requirements.txt**: Stores all the external dependencies in python required to run the game.
- **static/assets**: Images and fonts used in the frontend are stored in this directory.
- **static/index.html**: Serves as the entry point of the frontend, defining the DOM structure and integrating external styles and scripts.
- **static/styles.css**: Provides the styling layer for the frontend, managing layout, theming, and visual consistency across components.

- `static/logic.js`: Encapsulates the core client-side behaviour, including event handling, state updates, and real-time interaction logic.

7 Three Layer Communication

This project contains Three layers:

- The frontend (JavaScript + HTML5 Canvas)
- The backend (Python/Flask)
- Administration (Bash)

The three layers communicate as follows:

Browser(JS)→ POST `/save_score` or `/highscore.json`→Flask→`history.txt`←Bash.

Refer to section 5.8 to understand how the frontend communicates with the backend.

The bash script doesn't communicate with either of the frontend or the backend, it is unaware of the existence of the frontend or the backend. Refer to section 5.10 to understand how the administration works.

The format of logs in `history.txt` is chosen to be of the form shown with an example below:

```
27/04/2026, 20:27:59|Secreton|170|BODY|85.00s
```

This format was chosen to easily parse with `|` as the delimiter.

8 Design and Architectural Decisions

- The system follows a three-layer architecture consisting of a JavaScript-based frontend, a Flask backend, and a Bash-based administration layer. Each layer is designed to be independent, ensuring clear separation of concerns and ease of maintenance.
- The frontend handles all user interaction, rendering, and game logic using an HTML5 Canvas, allowing real-time updates and smooth gameplay. The backend, implemented using Flask, acts as an intermediary responsible for validating, processing, and persisting game data into `history.txt`. This ensures that the frontend remains lightweight and does not directly manage storage.
- The administration layer operates independently of both frontend and backend, interacting only with stored logs. This design choice enables offline analysis and management without affecting the running application.
- Game logic is further optimized by decoupling the game loop from snake movement, allowing dynamic speed adjustments for specific obstacles. Additionally, a simple text-based logging format with a fixed delimiter was chosen to enable efficient parsing using standard Unix tools such as `awk`, `sed`, and `sort`.
- Overall, the architecture prioritizes modularity, simplicity, and ease of extensibility while leveraging lightweight technologies across all layers.
- The `Food` and `Obstacle` classes are designed to be extensible, allowing new food items and obstacle types to be added easily without modifying existing logic.

9 Retrospection

9.1 Problems faced and how we overcame

- While implementing obstacles that modify the snake's speed, using a single frame rate for both rendering and movement led to inconsistencies. This was resolved by decoupling the game update rate from the snake's movement rate.
- During the implementation of immunity, collisions with walls required special handling. Since death checks originally occurred after movement, additional pre-collision checks were introduced to prevent unintended deaths and ensure consistent behaviour when the snake is immune.
- For short snake lengths (e.g., length 4), self-collision detection caused unintended behaviour. Adjustments to the movement and rendering timing were made to ensure consistent gameplay and eliminate discrepancies between game logic and visual updates.
- As the snake grew and occupied more grid space, spawning food and obstacles became increasingly constrained. To address this, spawn rates were adjusted to be inversely proportional (exponentially) to the number of available cells.
- Usernames containing special characters caused server-side issues. Input validation was introduced to restrict invalid characters and prevent crashes.
- While implementing advancements, a sliding animation was introduced to enhance visual feedback. This required positioning the element initially outside the visible screen area, then smoothly translating it into view to display the advancement, and subsequently moving it back out of the screen once the animation was complete.

9.2 What could have been done if we had 10 more hours

- Add sound effects to the game.
- Add more advancements in the game.
- Add better animations for the snake.

10 Bibliography and references

References

- [1] CSV Documentation. Python csv official documentation.
- [2] Flask Documentation. Flask official documentation, 2026.
- [3] MDN. Javascript fetch api documentaion.